

MACROFIT AI

Resumen del septimo bloque de trabajo

DIA 7 | Inicio de sesion real y autenticacion mediante JWT

Autor	Matias Gonzalez
Rama	feature/user-login
Commit	824065b - feat: implement user login with JWT
Estado final	Login probado y rama sincronizada

Objetivo del documento

Documentar la implementacion del inicio de sesion real, la verificacion segura de contraseñas, la generacion de tokens JWT y las pruebas realizadas con credenciales validas e invalidas.

1. Trabajo realizado

Durante el Dia 7 se completo el segundo flujo principal de autenticacion. Un usuario registrado puede iniciar sesion con su correo y contraseña, recibir un JWT firmado y utilizarlo posteriormente para acceder a endpoints protegidos.

- Integracion de feature/user-registration en develop.
- Creacion de la rama feature/user-login.
- Implementacion de verify_password() con pwplib y Argon2id.
- Instalacion y configuracion de PyJWT.
- Creacion de variables JWT seguras en .env y valores de ejemplo en .env.example.
- Creacion de create_access_token() con expiracion en UTC.
- Creacion del schema TokenResponse.
- Implementacion de login_user() en la capa service.
- Creacion del endpoint POST /auth/login.
- Pruebas desde Swagger con credenciales correctas e incorrectas.

Flujo implementado

POST /auth/login -> LoginRequest -> Service -> Repository -> verify_password -> JWT -> TokenResponse

2. Componentes del login

Componente	Responsabilidad
LoginRequest	Valida el correo y la contraseña recibidos.
auth.py	Expone POST /auth/login, obtiene la sesion y llama al service.
auth_service.py	Busca al usuario, valida las credenciales y solicita el token.
user_repository.py	Consulta PostgreSQL por correo electronico.
security.py	Verifica el hash Argon2id y crea el JWT.
TokenResponse	Devuelve access_token y token_type bearer.

Respuesta del login

```
{ "access_token": "eyJ...", "token_type": "bearer" }
```

Significado de Bearer y HS256

- Bearer indica como se envia el token en la cabecera Authorization.
- HS256 es el algoritmo utilizado para firmar el JWT.
- El token no incluye la contraseña ni password_hash.
- El identificador del usuario se guarda en el claim sub.
- La expiracion se guarda en exp y se calcula en UTC.

3. Seguridad del token

La configuracion sensible se mantiene en backend/.env, archivo ignorado por Git. La plantilla .env.example documenta las variables necesarias sin exponer la clave real.

Variable	Uso
JWT_SECRET_KEY	Clave privada para firmar y validar tokens.
JWT_ALGORITHM	Algoritmo de firma, configurado como HS256.
JWT_EXPIRE_MINUTES	Tiempo de validez del access token.

- La clave JWT se genero con el modulo secrets.
- La clave real no aparece en Git ni en .env.example.
- PyJWT genera el token como una cadena de texto.
- datetime.now(timezone.utc) evita problemas de zona horaria.
- El campo sub contiene user.id convertido a texto.

Verificacion de contraseña

verify_password(plain_password, hashed_password) devuelve True si la contraseña corresponde al hash y False en caso contrario. El hash nunca se descifra.

4. Pruebas realizadas

Escenario	Resultado	Comportamiento
Credenciales correctas	200 OK	Devuelve access_token y bearer.
Contraseña incorrecta	401 Unauthorized	Rechaza el acceso.
Correo inexistente	401 Unauthorized	Rechaza el acceso con el mismo mensaje.

- Se comprobo verify_password() con una contraseña correcta: True.
- Se comprobo verify_password() con una contraseña incorrecta: False.
- Se verifico que create_access_token() devuelve str y no un valor vacio.
- Swagger mostro POST /auth/login y el schema TokenResponse.
- No se compartio ni versiono la clave secreta real.

Mensaje de seguridad

Para correo inexistente y contraseña incorrecta se devuelve el mismo mensaje: "Correo electronico o contraseña incorrectos." Esto evita revelar que correos están registrados.

5. Sintaxis y aprendizajes

Concepto	Aprendizaje
Funciones Python	def define la funcion, : inicia el bloque y la indentacion sustituye a las llaves.
Tipos	parametro: tipo y -> tipo_devuelto documentan la funcion.
Diccionarios	Se utilizan pares "clave": valor para crear respuestas JSON.
Variables	Sin comillas se usa el valor de la variable; con comillas se devuelve texto literal.
Booleanos	if not password_is_valid ejecuta el bloque cuando el valor es False.
Imports	Los scripts del backend deben ejecutarse desde la carpeta backend para encontrar app.

Errores corregidos

- Ejecucion de Uvicorn desde la raiz: ModuleNotFoundError para app.
- Uso inicial de 202 Accepted en login: se dejo la respuesta automatica 200 OK.
- Uso de db:db en una llamada: se corrigio a db=db.
- Devolucion literal de "texto" en vez de la variable access_token.
- Import duplicado de PasswordHash.
- Espacios finales detectados mediante git diff --check.

6. Estado final y proximos pasos

- Inicio de sesion real completado.
- Contraseñas verificadas mediante Argon2id.
- JWT firmado y con expiracion.
- Respuesta Bearer validada desde Swagger.
- Commit 824065b creado y subido.
- Rama feature/user-login sincronizada y limpia.

Proximos pasos

- Actualizar el backlog y guardar este informe.
- Integrar feature/user-login en develop.
- Decodificar y validar JWT recibidos.
- Crear la dependencia get_current_user.
- Implementar GET /users/me como primer endpoint protegido.
- Añadir pruebas automatizadas para registro y login.
- Continuar con el perfil del usuario.

HITO: MacroFit AI ya permite registrarse e iniciar sesion con autenticacion JWT.